Übung 1

Da wir sie noch nicht in der Vorlesung behandelt haben und ich auch den Umgang mit Interfaces sehen möchte, verwenden Sie keine Lambda-Expressions oder Methodenreferenzen!

Packen Sie am Ende wieder alle Quelltextdateien in ein (bis drei) Archiv(e) und laden Sie sie hoch.

a) Kunden und Konten sortieren

Sorgen Sie dafür, dass die Klasse `Kunde` sortierbar ist. Schauen Sie sich den vorgegebenen Code der Klasse an, um zu entscheiden, wonach Kunden sortiert werden müssen.

Sortieren Sie das `Kunde`-Array im vorgegebenen Programm `KundenUndKontenSortieren` mit Hilfe der Methode `Arrays.sort`.

Sortieren Sie nun das `Konto`-Array dreimal mit der Methode `Arrays.sort`:

- einmal nach der Kontonummer, also in der für die Klasse Konto „natürlichen“ Ordnung,
- einmal nach dem Geburtstag des Kontoinhabers,
- das zweite Mal nach der IBAN des Kontos (der Benutzer gibt die Bankleitzahl ein, zu der die Konten gehören)

Tipp: Sehen Sie sich die Überladungen von `Arrays.sort` an. Das Interface `Comparator<Konto>` entspricht in etwa dem Vergleicher aus der Vorlesung.

b) Arrays füllen

Vorgegeben ist die Klasse `spielereien.EinigeArrays`.

Fügen Sie an jeder Stelle, wo ein Kommentar steht, einen Aufruf einer der folgenden Methoden aus der Klasse `Arrays` ein – das Programm sollte komplett ohne Schleifen auskommen:

- `public static void setAll(int[] einArray, IntUnaryOperator generator)`
  Die Methode ruft nacheinander für jedes Arrayelement aus `einArray` die im `generator` enthaltene Methode `applyAsInt(int)` auf, um das Arrayelement zu errechnen, und übergibt dabei den jeweiligen Index.
- `public static void setAll(double[] einArray, IntToDoubleFunction generator)`
  genauso für double-Werte

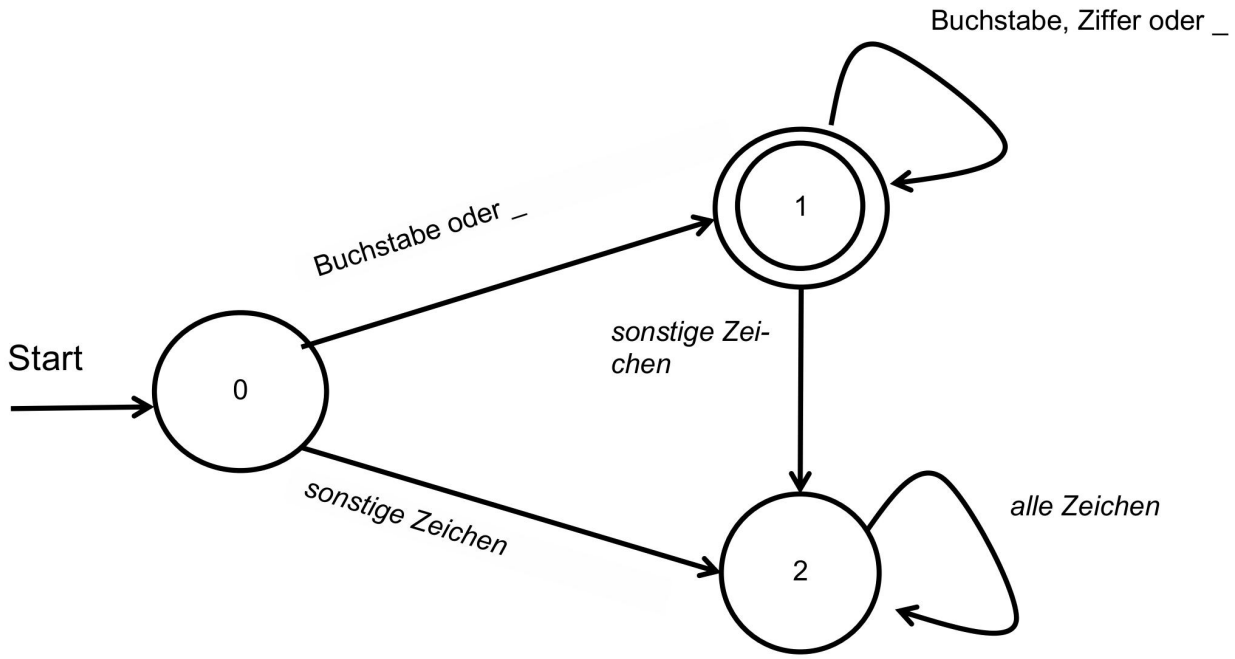
c) Endliche Automaten

Sie kennen alle Endliche Automaten. Entwickeln Sie ein Interface `automat.Endlicherautomat`, das einen Endlichen Automaten aus der Theoretischen Informatik beschreibt. Überlegen Sie sich für die grundlegenden Aktionen eines Endlichen Automaten geeignete Methodensignaturen, um einen Testablauf des Endlichen Automaten programmieren zu können.

Schreiben Sie eine default-Methode
`public boolean testen(String zeichenkette)`
Die Methode prüft, ob die `zeichenkette` vom Endlichen Automaten akzeptiert wird, d.h. sie

- startet den Automaten
- führt für jedes Zeichen in der Zeichenkette einen Zustandswechsel durch
- gibt zurück, ob er in einem akzeptierenden Zustand angekommen ist oder nicht.

Dorothea Hubrich
d_hubrich@web.de

Schreiben Sie die Klasse `automat.BezeichnerAutomat`. Sie soll gültige C-Variablennamen akzeptieren. Ein gültiger Name beginnt mit einem Buchstaben zwischen A und Z bzw. a und z oder einem Unterstrich. Danach sind wieder Buchstaben zwischen A und Z bzw. a und z, der Unterstrich oder Ziffern erlaubt (siehe Diagramm unten).

Zum Ausprobieren können Sie die vorgegebene Klasse `automat.Automatenspielerei` verwenden.